University of Technology, Jamaica

Faculty: Faculty of Engineering and Computing

School of Computing and Information Technology

Academic Year 2026-27 – Semester 1

Module: Web Programming

Module Code: CIT2011

Date of Submission: November 7, 2026

Name/s: Damani Williams

ID#: 2007591

Tutor: Mr. Sirisha Badhika

Title: Individual Assignment #2

## Table of Contents

Table of Contents ..... 2

IA#2: Dynamic Event Registration System ("Coding Conf") ..... 3

Validation Screenshots ..... 16

GitHub Repository Link ..... 21

Wireframe/s ..... 22

# IA#2: Dynamic Event Registration System ("Coding Conf")

**Course Title:** CIT2011 - Web Programming

**Assignment Due Date:** Week 9

**Technologies Required:** HTML5, CSS3, Native JavaScript (ES6+)

**Get IA#2 resources:**

## 1. Overview & Scenario

In this lab, you will act as a front-end developer building a responsive, multi-page client-side registration system for a fictitious tech conference called **"Coding Conf"**. The workflow moves a user from discovering event details to filling out a form, which dynamically outputs a stylized, customized digital ticket.

**Responsive Design**:

- Users may be on phones, tablets, laptops, desktops or large displays. Design content first; avoid assuming a single fixed screen width.
- Use flexible layouts with Flexbox/Grid, relative units and responsive media.
- Use Position, and Float properties.
- Include fluid typography
- CSS custom properties
- Test at multiple viewport sizes using DevTools'responsive mode. Include screenshots of test results
- Can a user read, tap, navigate and complete the task comfortably?

**Accessibility**:

- Accessible sites are designed so people with disabilities can interact meaningfully with the interface.
- Use semantic HTML before reaching for ARIA: headings, nav, main, buttons, labels and lists communicate structure.
- Ensure keyboard access, visible focus, readable contrast, useful alternative text and understandable forms.
- Accessibility is not a “final check”; it should influence HTML, CSS, JavaScript and design decisions from the start.

**Modern Selector Tools**:

- For example, :is(), :where(), :not() and :has() can reduce repetitive selectors.

You must implement this project using **ONLY** the features and data provided in the Week 1 - Week 6 material from the CIT2011 module. Strictly using semantic **HTML5**, custom **CSS3 layouts** (Flexbox/Grid), and **Vanilla JavaScript** for form parsing, form checking, client-side persistence, and rendering logic.

**2. System Architecture & Workflow Requirements**

Your web application directory must encompass three independent view states or pages:

1. **Landing Page (index.html):** A promotional launch page containing event headers, schedule metadata (Date: October 26, 2026; Location: Papine, Kgn), structural descriptions, and an actionable "Register Now" link or button leading to the form page.

2. **Registration / Login Page (login.html):** Contains the secure client form gathering user fields. It must process user inputs and handle client-side validations locally before advancing.
3. **Ticket Confirmation Page (ticket.html):** The final visual state displaying a dynamic dashboard containing a success confirmation text element alongside an advanced, glassmorphism-themed conference ticket populated by user parameters.

**Deductions** (Up to 5 marks per violation) for poor code formatting, chaotic indentations, missing documentation comments, unreferenced dead code loops, or copy-paste architecture omissions, broken links, missing accessibility features, etc.

**3. Technical & Engineering Specifications**

**A. Structural & Layout Integrity (HTML5 & CSS3)**

- **Semantic Architecture:** Build pages using clear layout components (<header>, <nav>, <main>, <section>, <footer>).
- **Aesthetic Themes & Effects:** Mimic a futuristic style leveraging dark purple space backgrounds, vibrant typography gradients, and custom modern visual rules. Do not link utility style packages or web frameworks (e.g., Tailwind CSS, Bootstrap).
- **Ticket Visual Matching:** The template on ticket.html must accurately lay out:
    - The master banner confirming standard textual messages ("Congrats, [Name]! Your ticket is ready.").
    - A premium ticket badge implementing glassmorphism characteristics (semi-transparent blurred backdrops, fine white bounds).

- Strategic arrangements showing full details: the matching "Coding Conf" branded title icon, specific metadata strings, a serial string tracker (e.g., #01609), the custom image profile picture, the parsed name string, and an accompanying GitHub profile link.

**B. Dynamic Logic & Form Engineering (JavaScript)**

- **State Persistence:** Transfer verified properties across execution windows seamlessly via standard browser interfaces like localStorage or sessionStorage.
- **Form Checking Controls:** Intercept form submissions inside login.html using event handlers. Programmatic rules must execute clean, contextual error labels to the DOM if criteria fail:
  - **Full Name:** Required field, cannot consist of empty spaces.
  - **Email ID:** Enforce strict structure checks matching explicit string regular expressions (user@domain.com).
  - **GitHub User Tag:** Enforce a standard user handle requirement beginning with the explicit @ prefix.
  - **Avatar Graphic Upload:** Validate presence and enforce system constraints validating explicit image types (JPEG/PNG) and size constraints (e.g., beneath 500KB).
- **Dynamic DOM Mutation:** During execution of ticket.html, query stored properties locally to populate your target elements dynamically. Implement boundary code blocks:

if direct route manipulation is attempted without state data present, immediately trigger client routing redirections forwarding the visitor back to index.html.

**4. Suggested Implementation Roadmap**

- **Architecture & Stylesheets:** Focus on component design. Wireframe and assemble structural markup templates across all views. Craft theme rules, asset containers, grid structures, responsive breakpoints, and glass design features.
- **Business Logic & Form Validation:** Focus on dynamic behaviors. Code form handlers, pattern filters, and validation rules. Setup persistence keys and read states securely during initialization of the ticket dashboard. Debug file parsers and complete rigorous execution tests.

**5. Project Submission Deliverables**

Compress your project workspace root inside a zipped archive structured as IDNo_LastName_FirstName_IA#2zip. The output tree must strictly mimic:

```
├── HTML
 └── index.html
 └── login.html
 └── ticket.html
├── css/
 └── style.css
├── js/ │
 ├── validation.js
 └── ticket.js
```

├── assets/

└── [Branding icons, imagery assets, background vectors]

**6. Evaluation & Grading Rubric (35 Marks Total)**

1. **Page Architecture & Deliverables**
    - Evaluates the presence, functional navigation workflow, and completeness of the three view states.
    - Landing Page (index.html) Layout: Features proper event headers, correct metadata strings (Date: October 26, 2026; Location: Papine, Kgn), structural overview text, and an active "Register Now" action node linking directly to the login state.
    - Registration Form Page (login.html): Securely displays a complete form capturing user fields (Full Name, Email, GitHub, Avatar Upload) with structural input containers, labels, and an intercepted submit event.
    - Ticket Confirmation View (ticket.html) Layout: Confirms standard textual messaging structures ("Congrats, [Name]! Your ticket is ready."), wraps elements cleanly inside an explicit glassmorphism container, and includes placeholders for logo, metadata, name, avatar, and GitHub handle.
    - PDF document that includes wireframe, student details, assignment details, screenshots, sample login details, link to github, free hosting link, etc.
2. **Technical Layout & CSS Style Engineering (40 marks Total)**

- Evaluates semantic structural entegrity, design aesthetics, responsive design, accessibility engineering, and advanced selector utilization.
- Solution must **ONLY** incorporate features and codes that were delivered for the CIT2011 Module for semester 1 academic year 2026-2027
- Excessive Ai usage will result in deduction of 50% of total overall grade.
- Creativity - usuage of transitions, animations, visual appeal, inclusion of useful and practical components that enhance the functionality of the final product. : **(5 marks)**

| **Category** | **Excellent (Full Marks)** | **Proficient (Minor Issues)** | **Developing (Partial Effort)** | **Unacceptable / Missing** |
|---|---|---|---|---|
| **HTML5 Structure & Semantics** *(5 Points)* | **5 marks:** Exemplary organization. Meaningful structural elements *(header, nav, main, section, footer)* are used comprehensively | **4 marks:** Clean architecture overall, but reveals minor semantic omissions or excessive general *div* wrappers where specific | **3-2 marks:** Layout shows weak semantic awareness. Elements fall back heavily onto generic structure | **1-0 marks:** Code fails standard syntax parsing, layout structure is completely broken, or |

| Category | Excellent (Full Marks) | Proficient (Minor Issues) | Developing (Partial Effort) | Unacceptable / Missing |
|---|---|---|---|---|
| | across all states. Meaningful form components, labels, and attributes. Markup passes strict layout parsing cleanly. | structural layout tags apply. | dividers without a structured taxonomy. | core files are missing from the workspace root. |
| **Responsive Design Implementation** *(5 Marks)* | **5 marks:** Stellar responsive engineering across mobile, tablet, and desktop. Fluid typography, CSS custom properties, and flexible grids/flexbox work cleanly. | **4 marks:** Responsive layouts work, but minor scaling quirks exist on narrow display viewports or typography sizes scale disproportionately. | **3-2 marks:** Relies heavily on fixed layout dimensions. Fails to implement CSS custom properties or fluid layouts | **1-0 marks:** Content is completely clipped or unreadable on mobile devices. No media queries, fluid properties, or |

| Category | Excellent (Full Marks) | Proficient (Minor Issues) | Developing (Partial Effort) | Unacceptable / Missing |
|---|---|---|---|---|
| | DevTools testing screenshots are included. | | smoothly across devices. | layout scaling present. |
| **Accessibility Compliance** *(5 Marks)* | **5 marks:** Explicit keyboard access, distinct visible focus styling, rich contrast ratios, alternative text (alt), and structural form labels are completely integrated. | **4 marks:** Strong semantic accessibility but exhibits minor gaps, such as missing form label connections or suboptimal icon description strings. | **3-2 marks:** Minimal accessibility consideration. Relies strictly on default browser interactions with un-labeled inputs or missing image alternative texts. | **1-0 marks:** Zero color contrast awareness, missing focus targets, or form inputs cannot be read by screen readers. |

| Category | Excellent (Full Marks) | Proficient (Minor Issues) | Developing (Partial Effort) | Unacceptable / Missing |
|---|---|---|---|---|
| **Modern Selector Tools** *(5 Marks)* | **5 marks:** Demonstrates direct structural utilization of modern functional pseudo-classes (e.g., :is(), :where(), :not(), or :has()) to effectively minimize stylesheet codebase repetitions. | **4 marks:** Modern functional pseudo-classes are used correctly, but only in isolated, basic selector paths. | **3-2 marks:** Minor formatting errors within functional selectors, or style rules revert to primitive cascading selector sequences. | **1-0 marks:** Completely ignores modern functional selectors. Relies exclusively on legacy repetitive multi-class paths. |
| **JS Validation & State** | **5 marks:** Form submittals are intercepted | **4 pts:** Validations fire successfully, but overlook niche | **3-2 pts:** Validation logic is | **11-0 pts:** Client scripts crash at |

| Category | Excellent (Full Marks) | Proficient (Minor Issues) | Developing (Partial Effort) | Unacceptable / Missing |
|---|---|---|---|---|
| **Management** *(5 marks)* | flawlessly via listeners. Performs custom regex string checks (Name present, strict user@domain.com matching, @ GitHub handle format, and image type/size validation under 500KB) with instant in-place DOM errors. Local/Session storage transfers data perfectly. | file stream variations, display alert fallback modals instead of dynamic in-place DOM labels, or possess minor state data parsing inefficiencies. | incomplete or easily bypassed. Regular expressions fail to parse fields securely, or data objects fail to pass between windows | initialization. Interception listeners are absent, or validation falls back exclusively onto native HTML constraint validation attributes. |

| Category | Excellent (Full Marks) | Proficient (Minor Issues) | Developing (Partial Effort) | Unacceptable / Missing |
|---|---|---|---|---|
| **Dynamic Ticket Generation** *(5 marks)* | **5 marks:** Fields parse properties smoothly on payload initialization. Renders file streams dynamically into active image profile elements. Script enforces secure routing boundary blocks: users attempting to view ticket.html directly without data are redirected | **4 marks:** Text data injects cleanly, but avatar image parsing is slightly glitchy or requires static local placeholders. Direct URL navigation redirection checks are missing or bypassed. | **3-2 marks:** Script populates minor pieces of string data. The target view depends heavily on static text template fields or hardcoded placeholder items. | **1-0 pts:** Script files fail to update the DOM at runtime, or console errors completely prevent the dashboard and ticket asset layout from mounting. |

| Category | Excellent (Full Marks) | Proficient (Minor Issues) | Developing (Partial Effort) | Unacceptable / Missing |
|---|---|---|---|---|
| | back to index.html instantly. | | | |
| **Code Cleanliness & Formatting** *(5 marks)* | **5 marks:** Production-grade files with consistent, uniform spacing rules, semantic attribute names, structured organization, and descriptive logical comments. | **4 marks:** Codes behave cleanly, but exhibit minor indent irregularities, tracking file clutter, or sparse comments near complex functional blocks. | **3-2 marks:** Code base features chaotic formatting, dead debugging lines, unreferenced variables, or cryptic tag naming patterns. | **1-0 pts:** Disorganized blocks lacking structure or containing un-refactored copy-paste code segments. |

**THE END**

# Validation Screenshots

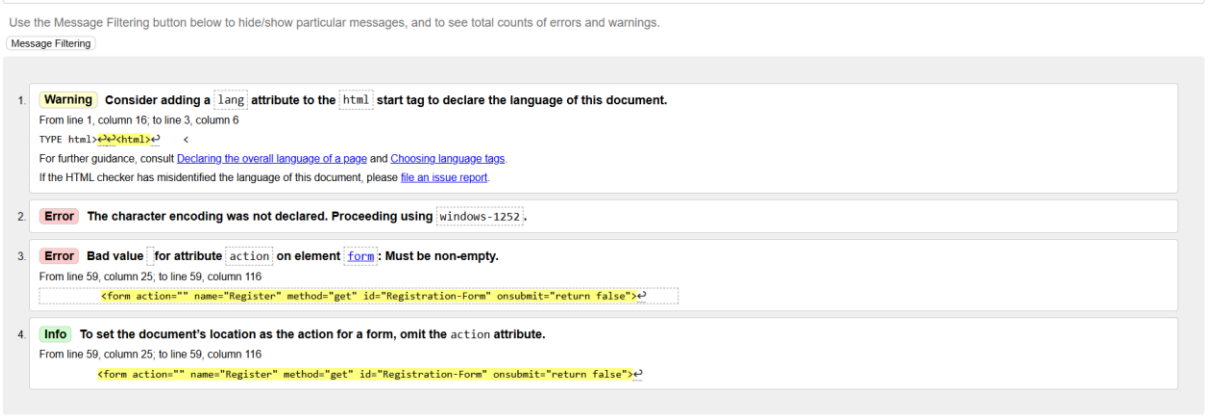



Figure 1: Showing validation for login.html by validator.w3.org

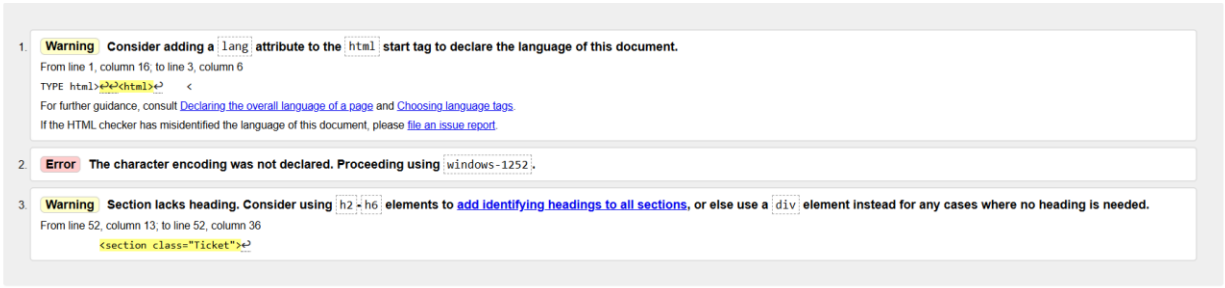



Figure 2: Showing validation for ticket.html by validator.w3.org

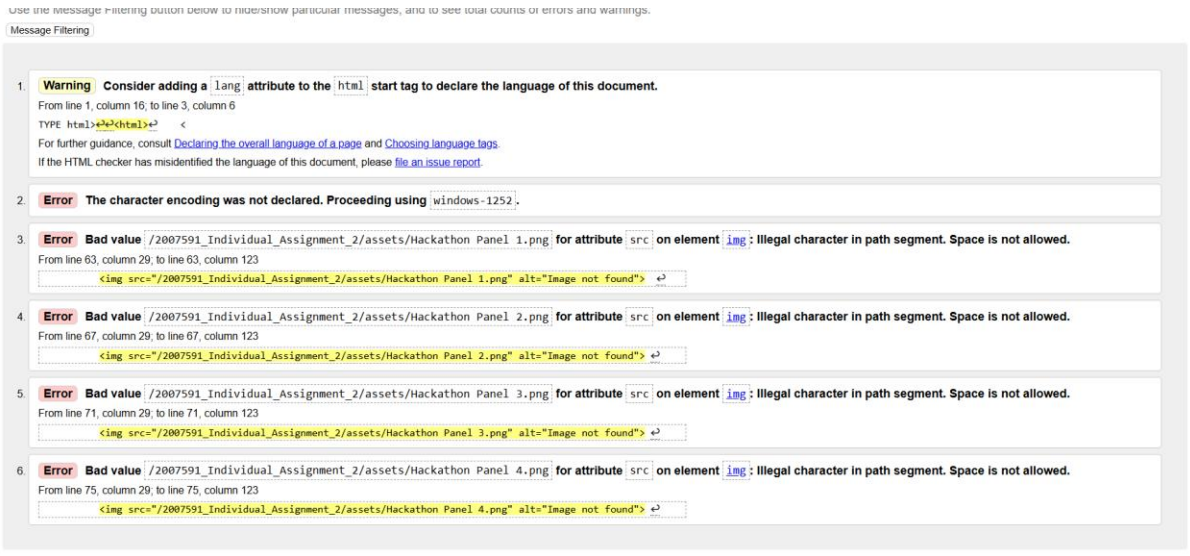
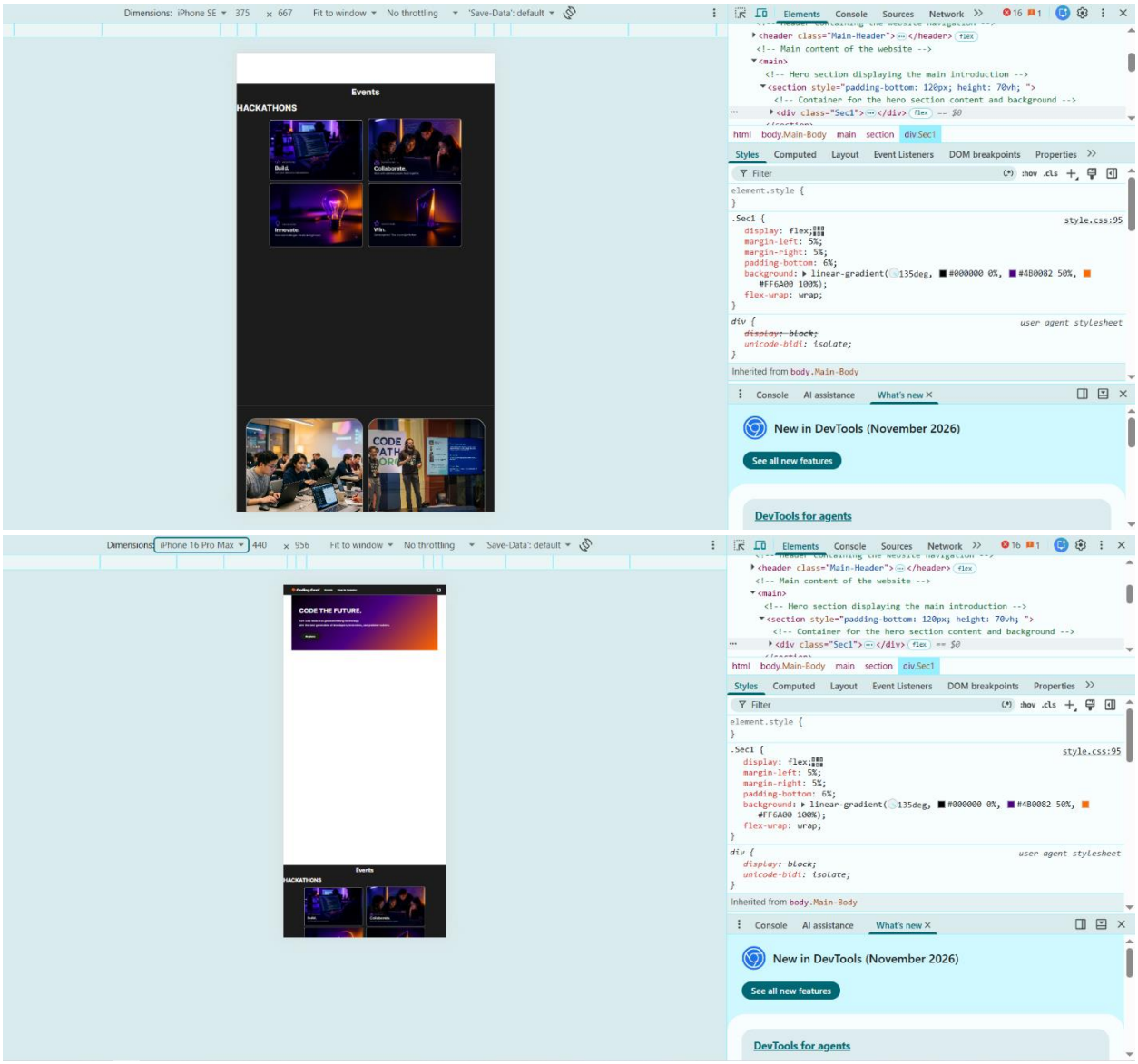
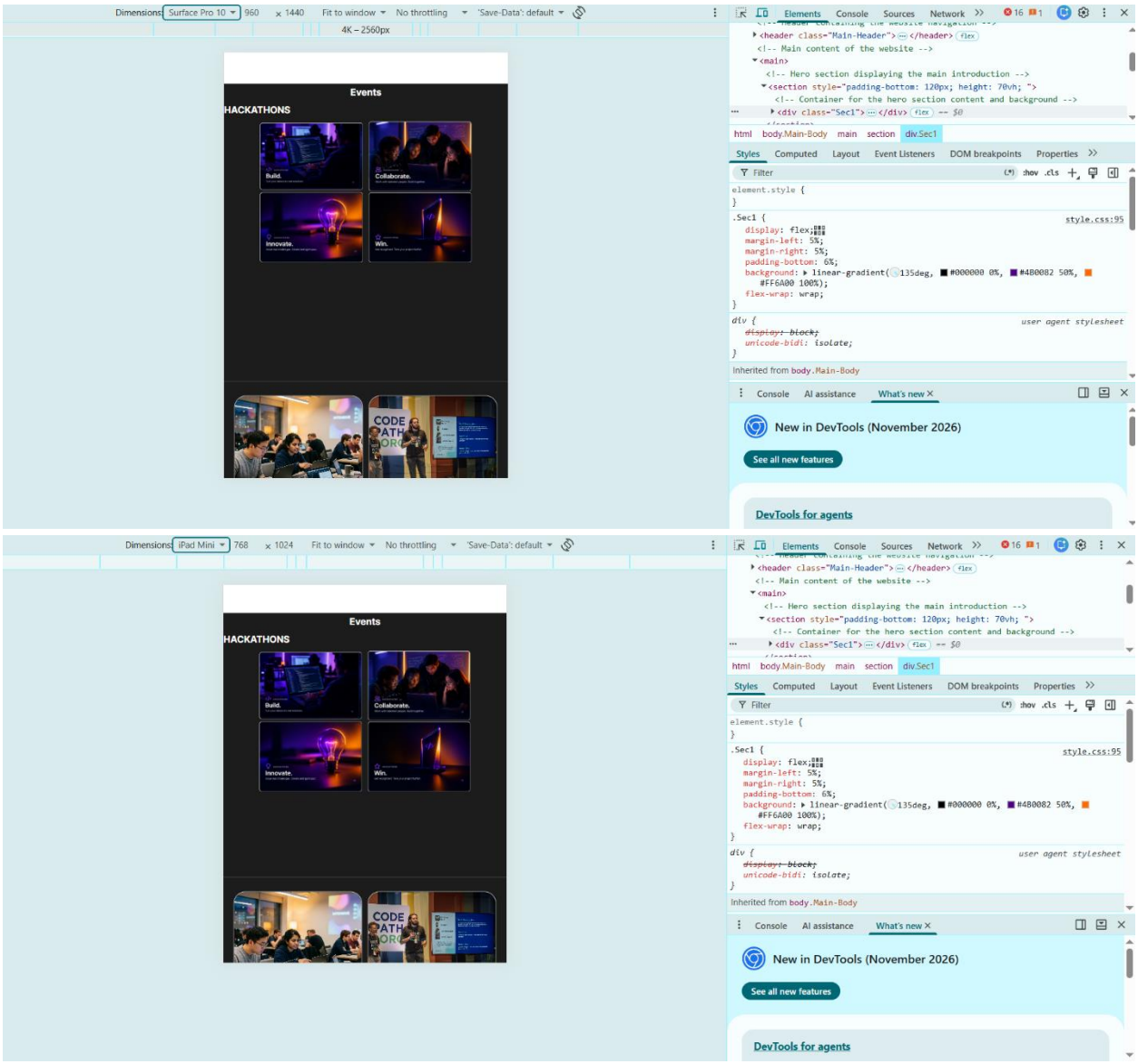


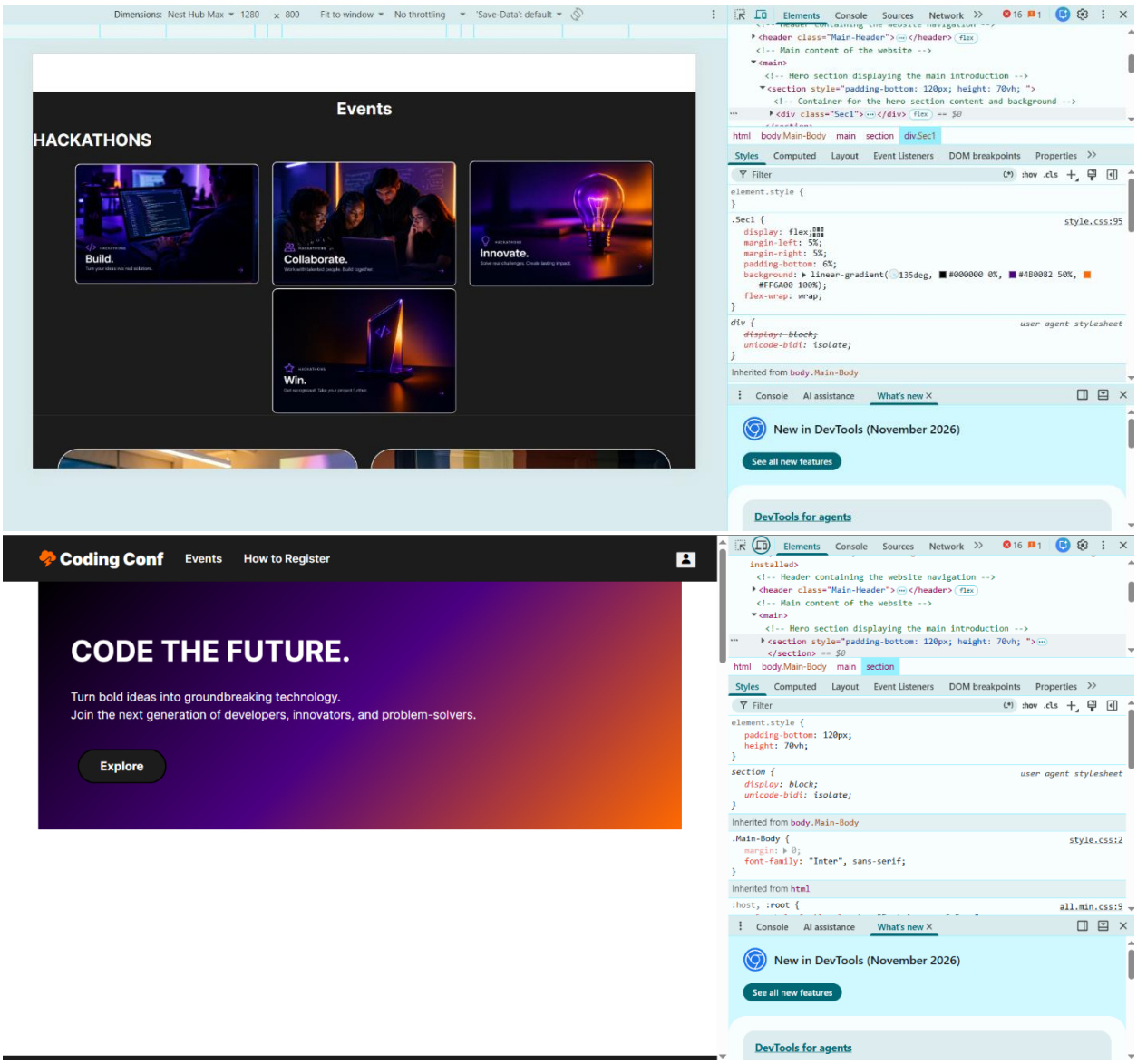

Figure 3: Showing validation for index.html by validator.w3.org




Figures 4 & 5: Showing viewport display size on mobile devices




Figures 6 & 7: Showing viewport display size on Tablets




Figures 8 & 9: Showing viewport display size on Desktop and Large Display

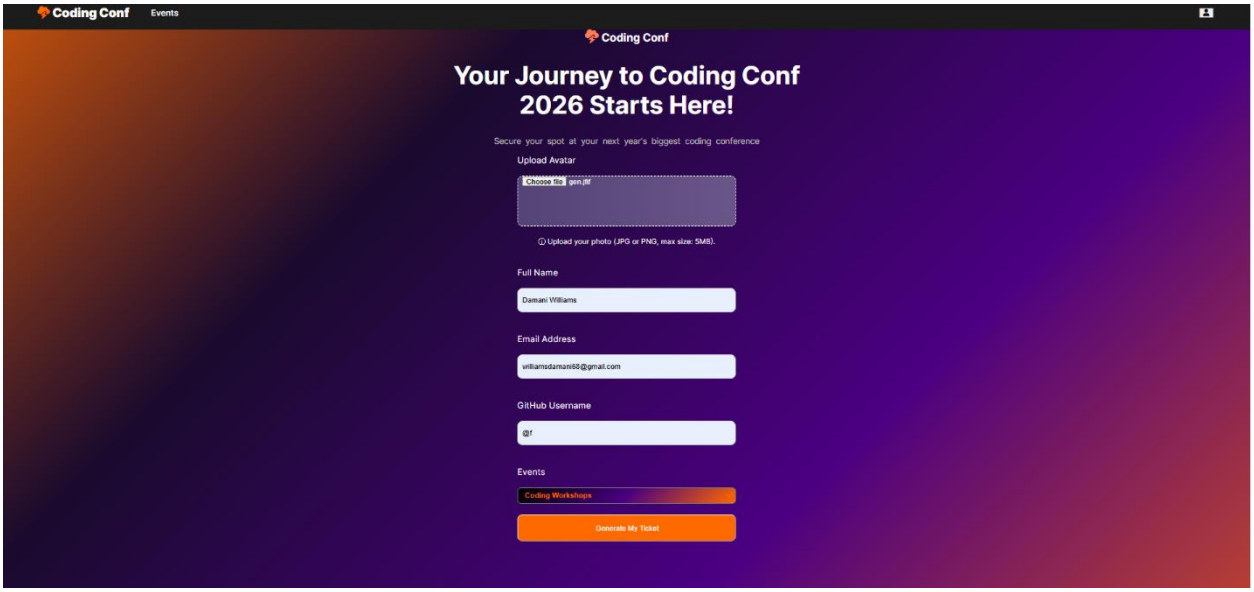



Figure 10: Showing correct sample login entry

## GitHub Repository Link

Link to my GitHub Repo: https://github.com/UTech-Damani/-2007591_Williams_Damani_IA-2

My IA#2 hosting link: https://utech-damani.github.io/-2007591_Williams_Damani_IA-2/

## Wireframe/s

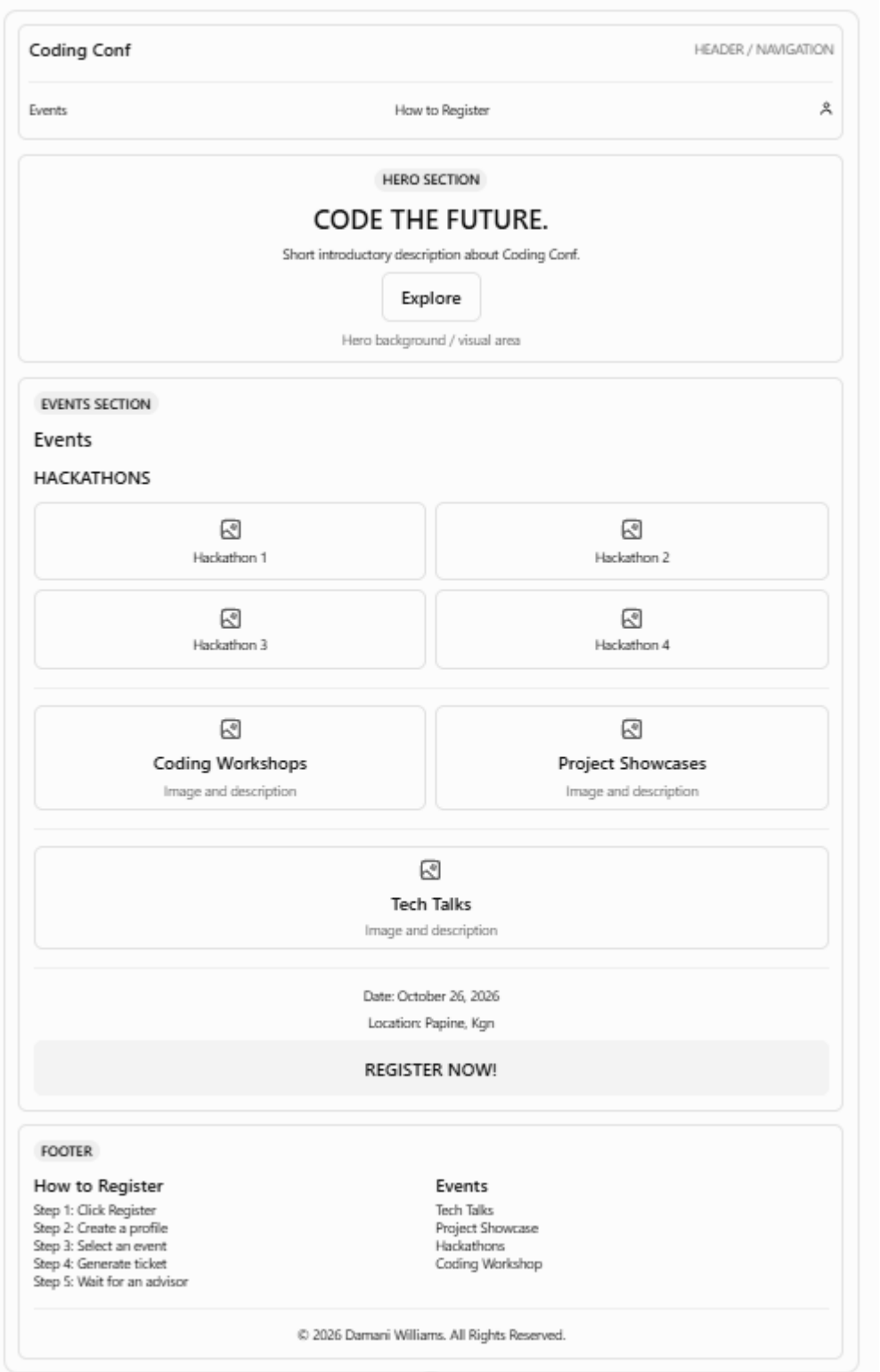



**Figure 11: Showing the wireframe for index.html**

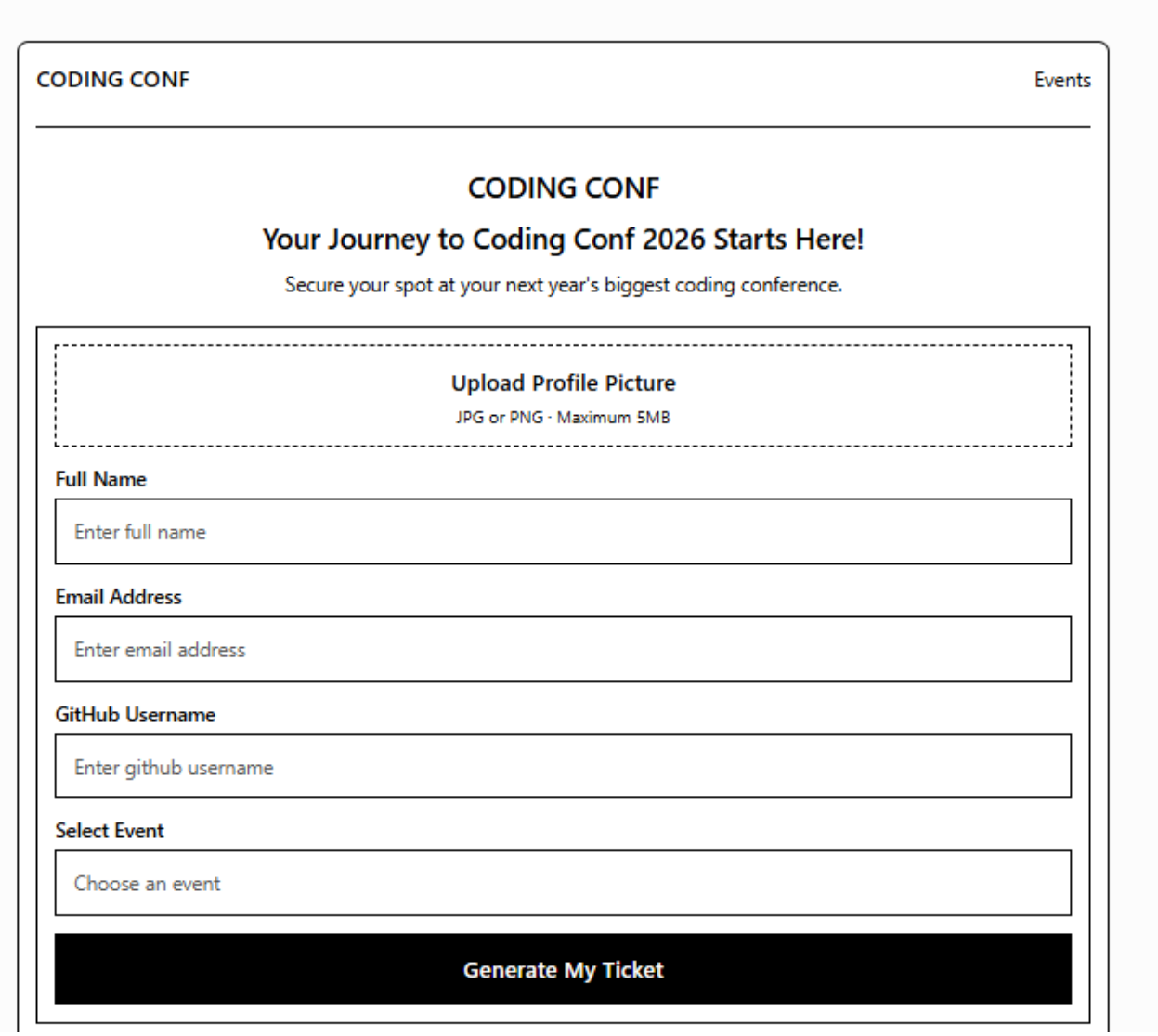



**Figure 11: Showing the wireframe for login.html**

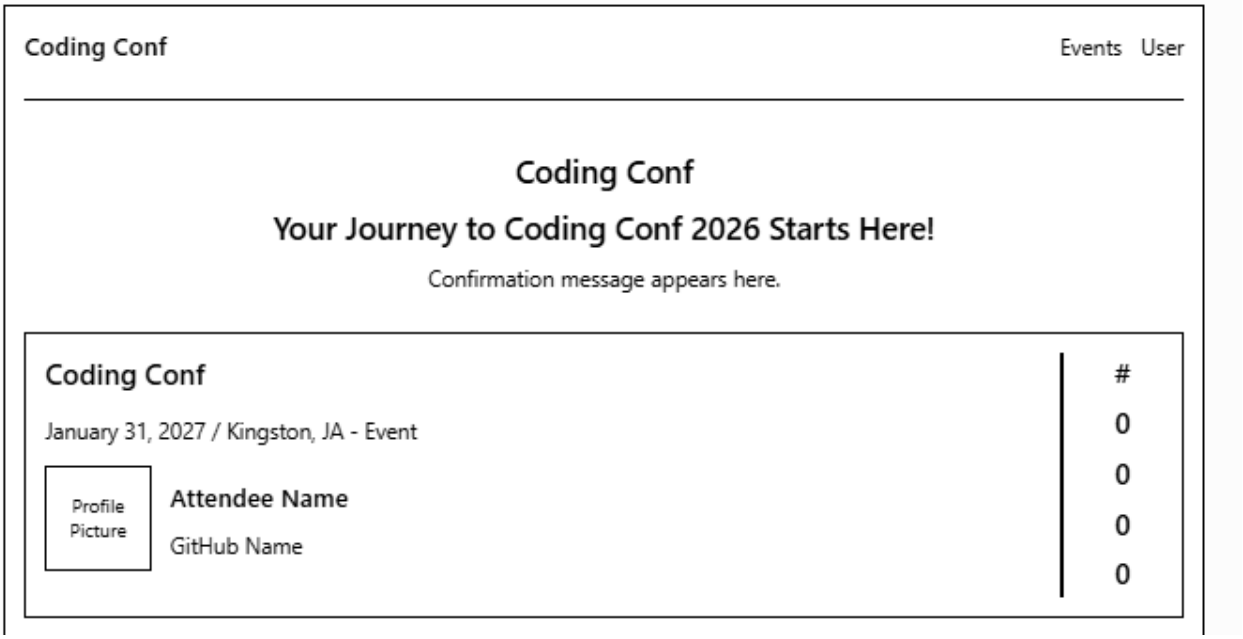



**Figure 11: Showing the wireframe for ticket.html**